09. 토큰 마이그레이션 메커니즘 분석

개요

OPinit의 토큰 마이그레이션 메커니즘은 L2에서 생성된 OP 토큰을 IBC 토큰으로 변환하여 다른 체인으로 전송할 수 있게 하는 시스템입니다. 이 메커니즘을 통해 L2 토큰이 IBC 생태계와 상호 운용성을 가질 수 있으며, 사용자는 L2 토큰을 다른 Cosmos 체인으로 자유롭게 이동시킬 수 있습니다.

핵심 컴포넌트

1. MigrationInfo 구조체

```
type MigrationInfo struct {
// L2 체인에서의 토큰 denomination
Denom  string  `protobuf:"bytes,1,opt,name=denom,proto3" json:"denom,omitempty"`
// L1 체인을 위해 L2 체인 상태에서 할당된 IBC 채널 ID
IbcChannelId  string  `protobuf:"bytes,2,opt,name=ibc_channel_id,json=ibcChannel
// L1 체인을 위해 L2 체인 상태에서 할당된 IBC 포트 ID
IbcPortId  string  `protobuf:"bytes,3,opt,name=ibc_port_id,json=ibcPortId,proto3
}
```

MigrationInfo 검증 로직:

```
func (m MigrationInfo) Validate() error {
```

```go
// IBC 채널 ID 검증
if err := host.ChannelIdentifierValidator(m.IbcChannelId); err != nil {
return errors.Wrap(sdkerrors.ErrInvalidRequest, err.Error())
}

// IBC 포트 ID 검증
if err := host.PortIdentifierValidator(m.IbcPortId); err != nil {
return errors.Wrap(sdkerrors.ErrInvalidRequest, err.Error())
}

// 토큰 denomination 검증
if err := sdk.ValidateDenom(m.Denom); err != nil {
return errors.Wrap(sdkerrors.ErrInvalidRequest, err.Error())
}

return nil
}
```

2. DenomPair 구조체

```go
type DenomPair struct {
// L2 토큰 denomination
Denom string `protobuf:"bytes,1,opt,name=denom,proto3" json:"denom,omitempty"`
```

```
// 기본(베이스) 토큰 denomination

BaseDenom  string  `protobuf:"bytes,2,opt,name=base_denom,json=baseDenom,proto3"

}
```

마이그레이션 정보 등록 프로세스

1. MsgRegisterMigrationInfo 메시지

메시지 구조:

```
type MsgRegisterMigrationInfo struct {

Authority  string  // 권한을 가진 주소 (일반적으로 거버넌스)

MigrationInfo MigrationInfo // 등록할 마이그레이션 정보

}
```

생성 함수:

```
func  NewMsgRegisterMigrationInfo(authority  string, migrationInfo MigrationInfo

return &MsgRegisterMigrationInfo{

Authority: authority,

MigrationInfo: migrationInfo,

}

}
```

검증 로직:

```go
func (msg MsgRegisterMigrationInfo) Validate(ac address.Codec) error {
// 권한 주소 검증
if _, err := ac.StringToBytes(msg.Authority); err != nil {
return err
}

// MigrationInfo 검증
if err := msg.MigrationInfo.Validate(); err != nil {
return err
}

return nil
}
```

2. RegisterMigrationInfo 핸들러 구현

```go
func (ms MsgServer) RegisterMigrationInfo(ctx context.Context, req *types.MsgReg
// 1. 메시지 검증
if err := req.Validate(ms.authKeeper.AddressCodec()); err != nil {
return nil, err
}

// 2. 권한 확인
```

```go
if ms.authority != req.Authority {
return nil, errorsmod.Wrapf(govtypes.ErrInvalidSigner, "invalid authority; expe
}

// 3. 중복 등록 방지
if migrationInfo, err := ms.Keeper.GetMigrationInfo(ctx, req.MigrationInfo.Deno
return nil, errorsmod.Wrapf(sdkerrors.ErrInvalidRequest, "migration info alread
}

// 4. 베이스 denomination 조회
baseDenom, err := ms.GetBaseDenom(ctx, req.MigrationInfo.Denom)
if err != nil {
return nil, err
}

// 5. 마이그레이션 정보 저장
if err := ms.Keeper.SetMigrationInfo(ctx, req.MigrationInfo); err != nil {
return nil, err
}

// 6. IBC denomination 계산 및 매핑 설정
ibcDenom := transfertypes.ParseDenomTrace(fmt.Sprintf("%s/%s/%s", req.MigrationI
if err := ms.Keeper.SetIBCToL2DenomMap(ctx, ibcDenom, req.MigrationInfo.Denom);
```

```go
return  nil, err
}

// 7. 이벤트 발생
sdkCtx := sdk.UnwrapSDKContext(ctx)
sdkCtx.EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeRegisterMigrationInfo,
sdk.NewAttribute(types.AttributeKeyFrom, req.Authority),
sdk.NewAttribute(types.AttributeKeyBaseDenom, baseDenom),
sdk.NewAttribute(types.AttributeKeyDenom, req.MigrationInfo.Denom),
sdk.NewAttribute(types.AttributeKeyIbcDenom, ibcDenom),
sdk.NewAttribute(types.AttributeKeyIbcChannelId, req.MigrationInfo.IbcChannelId)
sdk.NewAttribute(types.AttributeKeyIbcPortId, req.MigrationInfo.IbcPortId),
))

return &types.MsgRegisterMigrationInfoResponse{}, nil
}
```

토큰 마이그레이션 프로세스

1. MsgMigrateToken 메시지

메시지 구조:

```go
type MsgMigrateToken struct {
Sender  string  // 마이그레이션을 요청하는 사용자 주소
Amount sdk.Coin // 마이그레이션할 토큰 수량
}
```

생성 함수:

```go
func  NewMsgMigrateToken(sender  string, amount sdk.Coin) *MsgMigrateToken {
return &MsgMigrateToken{
Sender: sender,
Amount: amount,
}
}
```

검증 로직:

```go
func (msg MsgMigrateToken) Validate(ac address.Codec) error {
// 발신자 주소 검증
if  _, err := ac.StringToBytes(msg.Sender);  err != nil {
return  err
}

// 토큰 수량 검증 (양수이고 유효해야 함)
if !msg.Amount.IsValid() || !msg.Amount.IsPositive() {
```

```go
return ErrInvalidAmount
}

return nil
}
```

2. MigrateToken 핸들러 구현

```go
func (ms MsgServer) MigrateToken(ctx context.Context, req *types.MsgMigrateToken
// 1. 메시지 검증
if err := req.Validate(ms.authKeeper.AddressCodec()); err != nil {
return nil, err
}

// 2. 마이그레이션 정보 조회
migrationInfo, err := ms.Keeper.GetMigrationInfo(ctx, req.Amount.Denom)
if err != nil && errors.Is(err, collections.ErrNotFound) {
return nil, errorsmod.Wrapf(sdkerrors.ErrNotFound, "migration info not found")
} else if err != nil {
return nil, err
}

// 3. 발신자 주소 변환
```

```go
senderAddr, err := ms.authKeeper.AddressCodec().StringToBytes(req.Sender)
if  err != nil {
return  nil, err
}

// 4. 토큰 마이그레이션 실행
ibcCoin, err := ms.Keeper.MigrateToken(ctx, migrationInfo, senderAddr, req.Amoun
if  err != nil {
return  nil, err
}

// 5. 이벤트 발생
sdkCtx := sdk.UnwrapSDKContext(ctx)
sdkCtx.EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeMigrateToken,
sdk.NewAttribute(types.AttributeKeySender, req.Sender),
sdk.NewAttribute(types.AttributeKeyDenom, req.Amount.Denom),
sdk.NewAttribute(types.AttributeKeyAmount, req.Amount.Amount.String()),
sdk.NewAttribute(types.AttributeKeyMigratedCoin, ibcCoin.String()),
))

return &types.MsgMigrateTokenResponse{}, nil
```

```
}
```

3. 토큰 마이그레이션 핵심 로직

```
func (k Keeper) MigrateToken(ctx context.Context, migrationInfo types.MigrationI

// 1. 수량 검증

if !amount.IsPositive() {

return sdk.Coin{}, errorsmod.Wrap(sdkerrors.ErrInvalidRequest, "amount is not po

}


// 2. denomination 일치 확인

if  migrationInfo.Denom != amount.Denom {

return sdk.Coin{}, errorsmod.Wrap(sdkerrors.ErrInvalidRequest, "migration info d

}


// 3. 사용자로부터 모듈로 토큰 전송

err := k.bankKeeper.SendCoinsFromAccountToModule(ctx, sender, types.ModuleName,

if  err != nil {

return sdk.Coin{}, err

}


// 4. 모듈에서 토큰 소각

err = k.bankKeeper.BurnCoins(ctx, types.ModuleName, sdk.NewCoins(amount))
```

```go
if  err != nil {
return sdk.Coin{}, err
}

// 5. IBC 토큰 생성 및 발행
baseDenom, err := k.GetBaseDenom(ctx, amount.Denom)
if  err != nil {
return sdk.Coin{}, err
}
ibcCoin := sdk.NewCoin(ibcDenom(migrationInfo, baseDenom), amount.Amount)
err = k.bankKeeper.MintCoins(ctx, types.ModuleName, sdk.NewCoins(ibcCoin))
if  err != nil {
return sdk.Coin{}, err
}

// 6. IBC 토큰을 사용자에게 전송
err = k.bankKeeper.SendCoinsFromModuleToAccount(ctx, types.ModuleName, sender, s
if  err != nil {
return sdk.Coin{}, err
}

return  ibcCoin, nil
```

```
}
```

L1-L2 간 토큰 매핑 및 DenomPairs 관리

1. DenomPairs 저장소 관리

Keeper 구조체에서의 정의:

```
type Keeper struct {
// ... 다른 필드들
DenomPairs collections.Map[string, string] // L2 denom -> base denom
MigrationInfos collections.Map[string, types.MigrationInfo] // L2 denom -> migra
IBCToL2DenomMap collections.Map[string, string] // IBC denom -> L2 denom
}
```

초기화:

```
DenomPairs: collections.NewMap(sb, types.DenomPairPrefix, "denom_pairs", collect
```

2. 베이스 Denomination 조회

```
func (k Keeper) GetBaseDenom(ctx context.Context, denom  string) (string, error)
baseDenom, err := k.DenomPairs.Get(ctx, denom)
if  err != nil {
```

```
if errors.Is(err, collections.ErrNotFound) {
// DenomPair가 없으면 원본 denom을 베이스로 사용
return denom, nil
}
return "", err
}
return baseDenom, nil
}
```

3. IBC-L2 Denomination 매핑

매핑 설정:

```
func (k Keeper) SetIBCToL2DenomMap(ctx context.Context, ibcDenom, l2Denom strin
return k.IBCToL2DenomMap.Set(ctx, ibcDenom, l2Denom)
}
```

매핑 조회:

```
func (k Keeper) GetIBCToL2DenomMap(ctx context.Context, ibcDenom string) (strin
return k.IBCToL2DenomMap.Get(ctx, ibcDenom)
}
```

토큰 생성 함수 (IBC Denomination 계산)

1. IBC Denomination 생성 로직

```
func  ibcDenom(migrationInfo types.MigrationInfo, baseDenom  string) string {
// IBC 경로 형식: {port}/{channel}/{baseDenom}
denomTrace := fmt.Sprintf("%s/%s/%s", migrationInfo.IbcPortId, migrationInfo.Ibc
return  transfertypes.ParseDenomTrace(denomTrace).IBCDenom()
}
```

2. 마이그레이션된 토큰 입금 처리

```
func (k Keeper) HandleMigratedTokenDeposit(ctx context.Context, sender sdk.AccAd
// 1. 수량 검증
if !ibcCoin.IsPositive() {
return sdk.Coin{}, errorsmod.Wrap(sdkerrors.ErrInvalidRequest, "amount is not po
}

// 2. L2 denomination 조회
l2Denom, err := k.GetIBCToL2DenomMap(ctx, ibcCoin.Denom)
if  err != nil {
return sdk.Coin{}, err
}

// 3. IBC 토큰을 모듈로 전송
```

```go
err = k.bankKeeper.SendCoinsFromAccountToModule(ctx, sender, types.ModuleName, s
if  err != nil {
return sdk.Coin{}, err
}


// 4. IBC 토큰 소각
err = k.bankKeeper.BurnCoins(ctx, types.ModuleName, sdk.NewCoins(ibcCoin))
if  err != nil {
return sdk.Coin{}, err
}


// 5. L2 토큰 발행
l2Coin := sdk.NewCoin(l2Denom, ibcCoin.Amount)
err = k.bankKeeper.MintCoins(ctx, types.ModuleName, sdk.NewCoins(l2Coin))
if  err != nil {
return sdk.Coin{}, err
}


// 6. L2 토큰을 사용자에게 전송
err = k.bankKeeper.SendCoinsFromModuleToAccount(ctx, types.ModuleName, sender, s
if  err != nil {
return sdk.Coin{}, err
}
```

```
// 7. 브리지 훅 처리 (메모가 있는 경우)
var  migratedTokenDepositMemo ophosttypes.MigratedTokenDepositMemo
if  err := json.Unmarshal([]byte(memo), &migratedTokenDepositMemo);  err != nil
return  l2Coin, nil
}

if  len(migratedTokenDepositMemo.OPinit) > 0 {
params, err := k.GetParams(ctx)
if  err != nil {
return sdk.Coin{}, err
}

if  success, reason := k.handleBridgeHook(sdk.UnwrapSDKContext(ctx), migratedTok
return sdk.Coin{}, errorsmod.Wrap(sdkerrors.ErrInvalidRequest, reason)
}
}

return  l2Coin, nil
}
```

마이그레이션된 토큰 출금 처리

1. 출금 시 마이그레이션 자동 처리

```
func (k Keeper) HandleMigratedTokenWithdrawal(ctx context.Context, msg *types.Ms

denom := msg.Amount.Denom

// 1. 마이그레이션 정보 확인

migrationInfo, err := k.GetMigrationInfo(ctx, denom)

if err != nil && errors.Is(err, collections.ErrNotFound) {

return false, nil // 마이그레이션 정보가 없으면 일반 출금으로 처리

} else if err != nil {

return false, err

}


// 2. 발신자 주소 변환

sender, err := k.addressCodec.StringToBytes(msg.Sender)

if err != nil {

return false, err

}


// 3. OP 토큰을 IBC 토큰으로 마이그레이션

ibcCoin, err := k.MigrateToken(ctx, migrationInfo, sender, msg.Amount)

if err != nil {

return false, err

}
```

```
// 4. IBC 전송 메시지 생성

transferMsg := transfertypes.NewMsgTransfer(

migrationInfo.IbcPortId,

migrationInfo.IbcChannelId,

ibcCoin,

msg.Sender,

msg.To,

clienttypes.NewHeight(0, 0),

uint64(sdk.UnwrapSDKContext(ctx).BlockTime().UnixNano())+transfertypes.DefaultRe

"forwarded from opchild module",

)



// 5. IBC 전송 실행

sdkCtx := sdk.UnwrapSDKContext(ctx)

if  handler := k.msgRouter.Handler(transferMsg);  handler == nil {

return false, errorsmod.Wrap(sdkerrors.ErrNotFound, sdk.MsgTypeURL(transferMsg))

} else  if  res, err := handler(sdkCtx, transferMsg);  err != nil {

return false, err

} else {

sdkCtx.EventManager().EmitEvents(res.GetEvents())

}
```

```
return true, nil

}
```

상태 관리 및 조회 함수

1. MigrationInfo 관리

```
// 마이그레이션 정보 설정

func (k Keeper) SetMigrationInfo(ctx context.Context, migrationInfo types.Migrat

return  k.MigrationInfos.Set(ctx, migrationInfo.Denom, migrationInfo)

}


// 마이그레이션 정보 조회

func (k Keeper) GetMigrationInfo(ctx context.Context, denom  string) (types.Migr

return  k.MigrationInfos.Get(ctx, denom)

}


// 마이그레이션 정보 존재 확인

func (k Keeper) HasMigrationInfo(ctx context.Context, denom  string) (bool, erro

return  k.MigrationInfos.Has(ctx, denom)

}


// 모든 마이그레이션 정보 순회
```

```go
func (k Keeper) IterateMigrationInfos(ctx context.Context, cb func(denom strin
return k.MigrationInfos.Walk(ctx, nil, func(denom string, migrationInfo types.
if stop, err := cb(denom, migrationInfo); err != nil {
return true, err
} else if stop {
return true, nil
}
return false, nil
})
}
```

이벤트 시스템

1. 마이그레이션 정보 등록 이벤트

```go
sdk.NewEvent(
types.EventTypeRegisterMigrationInfo,
sdk.NewAttribute(types.AttributeKeyFrom, req.Authority),
sdk.NewAttribute(types.AttributeKeyBaseDenom, baseDenom),
sdk.NewAttribute(types.AttributeKeyDenom, req.MigrationInfo.Denom),
sdk.NewAttribute(types.AttributeKeyIbcDenom, ibcDenom),
sdk.NewAttribute(types.AttributeKeyIbcChannelId, req.MigrationInfo.IbcChannelId)
sdk.NewAttribute(types.AttributeKeyIbcPortId, req.MigrationInfo.IbcPortId),
```

```
)
```

2. 토큰 마이그레이션 이벤트

```
sdk.NewEvent(
types.EventTypeMigrateToken,
sdk.NewAttribute(types.AttributeKeySender, req.Sender),
sdk.NewAttribute(types.AttributeKeyDenom, req.Amount.Denom),
sdk.NewAttribute(types.AttributeKeyAmount, req.Amount.Amount.String()),
sdk.NewAttribute(types.AttributeKeyMigratedCoin, ibcCoin.String()),
)
```

보안 고려사항

1. 권한 관리

- 마이그레이션 정보 등록은 거버넌스 권한이 필요
- 중복 등록 방지를 통한 무결성 보장
- 유효한 IBC 채널/포트 ID 검증

2. 토큰 무결성

- 마이그레이션 시 1:1 토큰 교환 보장
- 소각과 발행의 원자적 실행
- 잔액 부족 시 트랜잭션 실패

3. 에러 처리

- 마이그레이션 정보 미등록 시 적절한 에러 반환
- IBC 전송 실패 시 롤백 메커니즘
- 유효하지 않은 denomination에 대한 검증

사용 시나리오

1. 일반적인 마이그레이션 플로우

1. **마이그레이션 정보 등록**: 거버넌스를 통해 특정 L2 토큰에 대한 IBC 채널 정보 등록
2. **토큰 마이그레이션**: 사용자가 L2 토큰을 IBC 토큰으로 변환
3. **IBC 전송**: 변환된 IBC 토큰을 다른 체인으로 전송
4. **역방향 마이그레이션**: 다른 체인에서 돌아온 IBC 토큰을 다시 L2 토큰으로 변환

2. 출금 시 자동 마이그레이션

1. **출금 요청**: 사용자가 마이그레이션 가능한 토큰으로 출금 요청
2. **자동 마이그레이션**: 시스템이 자동으로 토큰을 IBC 토큰으로 변환
3. **IBC 전송**: 변환된 토큰을 목적지 체인으로 직접 전송

이 토큰 마이그레이션 메커니즘을 통해 OPinit L2는 IBC 생태계와의 완전한 상호 운용성을 제공하며, 사용자는 L2 토큰을 Cosmos 생태계 전반에서 자유롭게 사용할 수 있습니다.